Digital Systems

Lecture 6

Subroutines

Mark van der Wilk

Hilary Term 2025

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

Previously

- Could not call other subroutines.
-

Previously

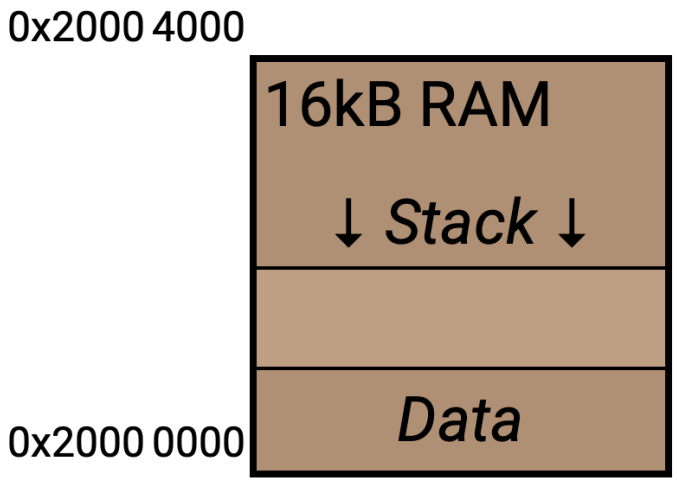
- Could not call other subroutines.
- Could only use `r0` - `r3`, while leaving `r4` and above unchanged.

- Let subroutines use memory relative to the *stack pointer* (register `sp`).
-
-

- Let subroutines use memory relative to the *stack pointer* (register `sp`).
- Memory below in the stack is used by calling subroutines, and should remain untouched.
-

- Let subroutines use memory relative to the *stack pointer* (register `sp`).
- Memory below in the stack is used by calling subroutines, and should remain untouched.
- Memory above in the stack is free to be used by the subroutine.

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`

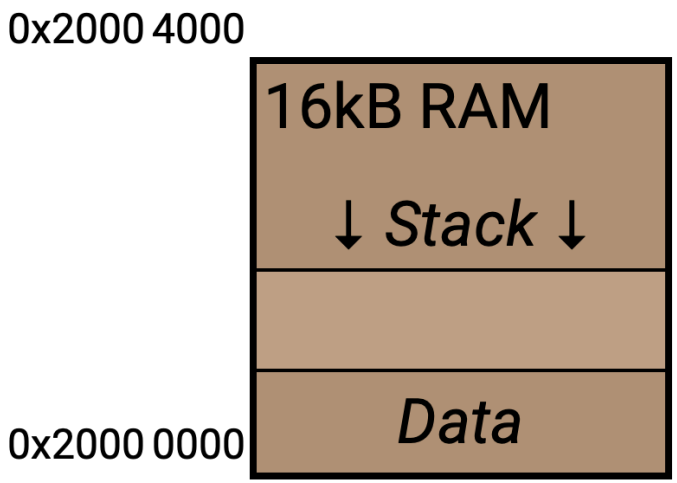
Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

- Convention: Stack starts at top of address space, and grows downwards. So “above” in the stack has a lower address!
-

•

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`

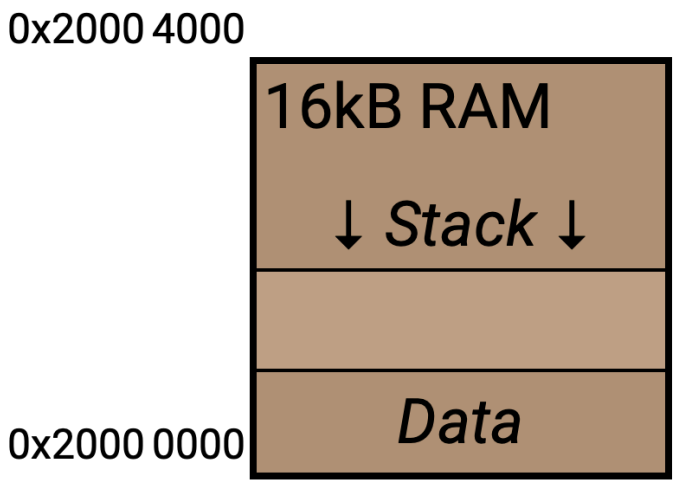
Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

- Convention: Stack starts at top of address space, and grows downwards. So "above" in the stack has a lower address!
- Each time a subroutine is called, `sp` points to a usable region of memory (stack frame), that won't interfere with other subroutines.

•

The Stack



Example:

`main()`
`sub1()`
`sub2()`
`sub3()`

Return:

`main()`
`sub1()`
`sub2()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`

Call next:

`main()`
`sub1()`
`sub2()`
`sub4()`
`sub5()`

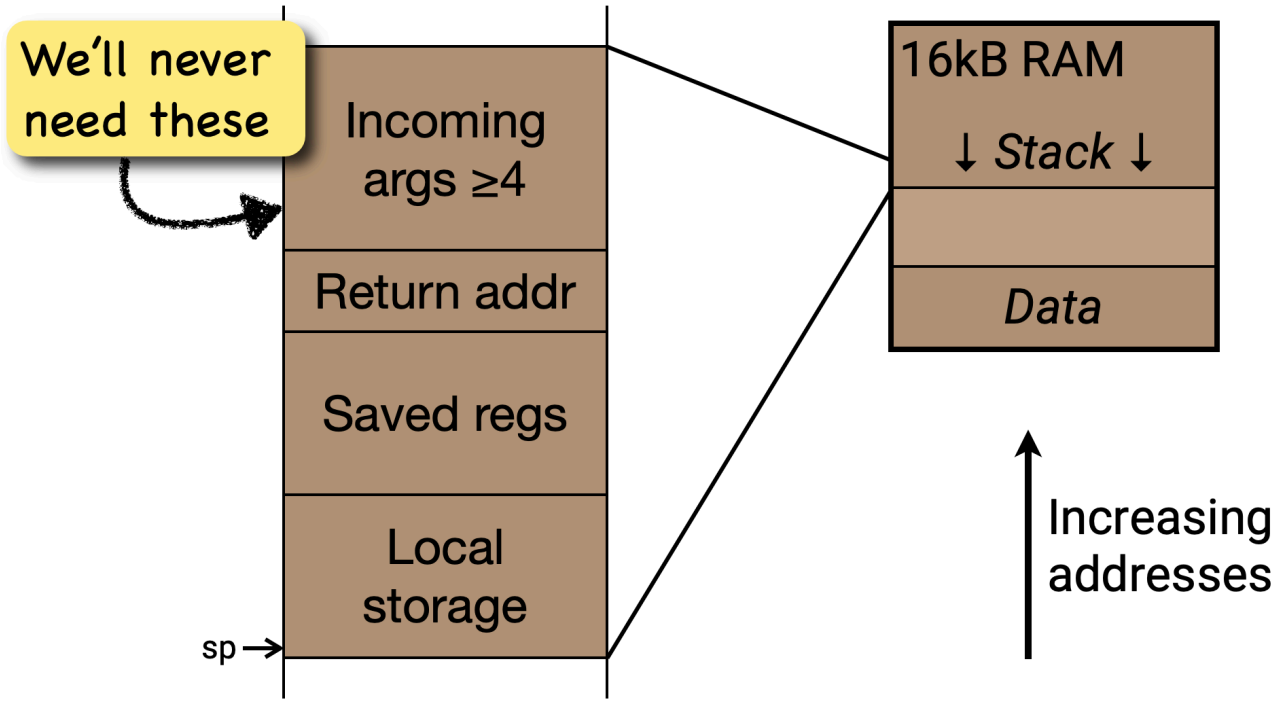
- Convention: Stack starts at top of address space, and grows downwards. So "above" in the stack has a lower address!
- Each time a subroutine is called, `sp` points to a usable region of memory (stack frame), that won't interfere with other subroutines.

- When a subroutine returns, the caller finds its stack frame as it left it.

Subroutine Contract

Subroutine Convention

Subroutine can use stack in whatever way it wants (assuming there is space!), but ARM provides instructions for:



Storing Registers

- `push {REGISTER_LIST}`: Place registers in list on stack in order, and decrease `sp` accordingly.
-

Storing Registers

- `push {REGISTER_LIST}`: Place registers in list on stack in order, and decrease `sp` accordingly.
- `pop {REGISTER_LIST}`: Take values from stack, and place into the registers in list order, and increase `sp` accordingly.

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0` - `r7`)
-
-

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
-

Push & Pop Peculiarities

- `push`/`pop` can store/restore lower registers (`r0 - r7`)
- `push` can store `lr`, as well
- `pop` can restore to `pc`, as well

Putting it all together

Example: Factorial

```
int fac(int n) {
    int k = n, f = 1;
    while (k != 0) {
        f = mult(f, k);
        k = k-1;
    }
    return f;
}
```

Subroutine calls another subroutine! So we will use what we discussed.

Example: Factorial, opening

- As usual, let's start by converting code to assembly, and not worrying about efficiency.
- For clarity, I recommend making clear which variables are assigned to which registers (little table in your exam answer helps!)
- Let's assign `k` to `r4`, and `f` to `r5` (logic behind this?)
- Must use `push`.

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
```

Example: Factorial, Loop

Again, we can worry about optimisations later:

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished
```

Example: Factorial, Subroutine Call

Again, we can worry about optimisations later:

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0
```

Example: Factorial, Finish Loop

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0

    subs r4, r4, #1             @ Decrement k
    b again                     @ And repeat
```

Example: Factorial, Return Subroutine

```
...
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

...

finish:
    movs r0, r5                 @ Return f
    pop {r4, r5, pc}
```

Example: Factorial, All Together

```
fac:
    push {r4, r5, lr}
    movs r4, r0                 @ Initialise k to n
    movs r5, #1                 @ Set f to 1
again:
    cmp r4, #0                  @ Is k = 0?
    beq finish                  @ If so, finished

    movs r0, r5                 @ Set f to f * k
    movs r1, r4
    bl mult                     @ branch link instruction!
    movs r5, r0
```

```
    subs r4, r4, #1          @ Decrement k
    b again                  @ And repeat
finish:
    movs r0, r5              @ Return f
    pop {r4, r5, pc}
```

Example: Factorial, optimisation

You may have spotted that `f` can live in `r0` constantly, since the return value of `mult()` will also be the first argument for the next call.

Optimising compilers can spot this.

Summary